

导航初识

Tasks

感谢sp和火锅战队导航组视频相关介绍，其图片内容来自sp27培训

导航的任务基本上分为：定位——感知——规划——控制——行为

- 定位：`localization`，通过传感器并进行观测获取整车位置姿态，这里不强调非常精细的姿态，一般这个姿态指的是云台yaw的姿态
 - 定位方面：一般利用 `lio` 算法，这里选择的是 `point_lio`，输出的是车体上电位姿 (`lidar_`)odom 2车体现在的位姿 `base_link` 和速度 (?)，同时会保存一份 PCD点云地图
 - 局限：就是只要是传感器，只要有噪声，只要是设计的观测器算法，就是会漂 (

02 • LIO

Point-LIO

Point-LIO 出自香港大学 (HKU) 机器人实验室, Point-LIO: Robust High-Bandwidth LiDAR-Inertial Odometry. 它是一套鲁棒、高带宽的激光雷达 - 惯性里程计, 能够估计机器人极端剧烈运动状态。

- 激光雷达 + IMU 紧耦合, 连续估计相对运动
- 输出高频位姿 (`lidar_odom` → `base_link`) + 速度
- 输出一份 PCD 点云

局限: 只知道“相对起点走了多远”, 跑久了会漂, 不知道自己在全局地图的哪。

- 重定位：一般的导航栈都分为——`map2odom2baselink`，重定位就是维护 `map2odom` 这段变化以修正定位的误差
 - `map2odom`：初始一般置成 `0 0 0 0 0 0(x y z r p y)`，后续通过重定位去维护，这里主流的重定位就是 `ICP` 及其衍生算法 (局限就是需要在线建图)
 - 注意：这个只是补偿误差不一定是真实的 `map2odom`

如何纠偏

1、初始时，车在map系原点：

- lidar_odom $\rightarrow$ base_link: (0,0) (手动指定)
- 重定位: map $\rightarrow$ lidar_odom: (0,0) (还没偏)
- 合成 map $\rightarrow$ base_link: (0,0) $\checkmark$

2、开了一会儿，LIO 飘了：

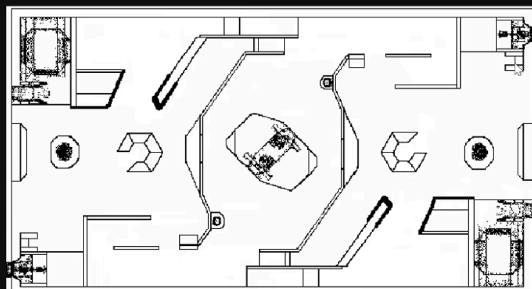
- 车其实已经到了地图上的 (5,0)，但 LIO 积分偏了，觉得自己到了 (5.3,0)
- lidar_odom $\rightarrow$ base_link: (5.3,0) $\leftarrow$ LIO 自己的估计 (偏了 +0.3)
- 若没有重定位，还当 map = lidar_odom，规划器会以为车在 (5.3,0) $\times$

3、重定位纠偏：

- GICP 拿当前点云去对先验地图，发现整棵 lidar_odom 树相对地图往 +x 多偏了 0.3，于是发布 map $\rightarrow$ lidar_odom: (-0.3,0)
- 合成 map $\rightarrow$ base_link: (0,0) $\checkmark$

- 感知：其实简单来说就是从机器人的视角来看哪些地方能走哪些不能走，一般就是高程分析和代价设计
 - 全局栅格地图：一般静态离线生成，或者在线建图生成
 - 全局/局部代价：全局代价直接由全局栅格代价设计而来，局部代价还需要实时的点云数据和分析并进行代价设计；全局代价一般是用于给出全局路径规划，局部代价一般用于精细化的局部规划和避障

占据栅格地图

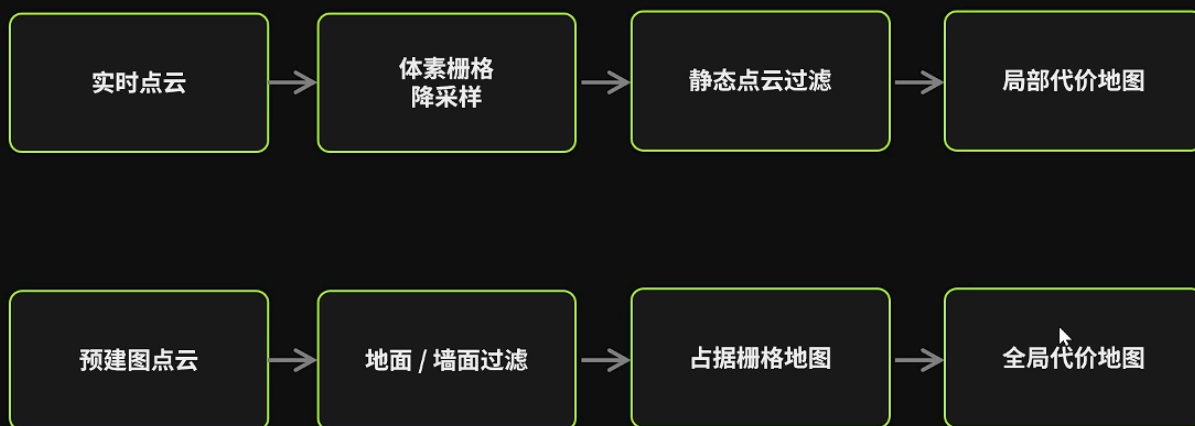


```
image: rmuc_map.pgm
resolution: 0.05
origin: [-0.881000, -0.550656, 0.0]
negate: 0
occupied_thresh: 0.65
free_thresh: 0.196
```

$$\text{占用概率 } p = 1 - \frac{\text{像素}}{255}$$

yaml	含义
$p \geq \text{occupied_thresh}$	占据
$p \leq \text{free_thresh}$	空闲
$\text{free_thresh} < p < \text{occupied_threh}$	未知

过滤：把原始点云变成“哪里不能走”



个人认为：感知是整个导航中最难的一部分，原因就是将一个2D导航任务变成有3D感知的2.5D导航，所以，对于相关3D感知的设计非常关键，一般3D感知用于局部代价地图相关（避障和一些跨越特殊地形），至于为什么不用于全局，主要是全局相关的代价地图是可以进行离线静态设计以规划出比赛场上想要的路线，但是局部相关结合了自身车和场上其他车的一些情况，是在线的，因而用到了感知（高德地图——全局代价和规划；人眼识别并前进——局部代价和跟随）



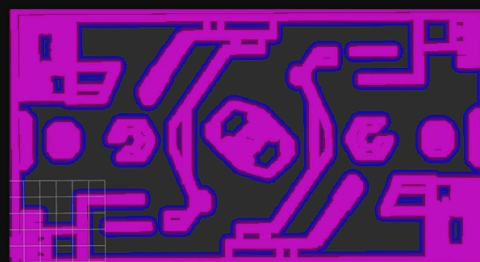
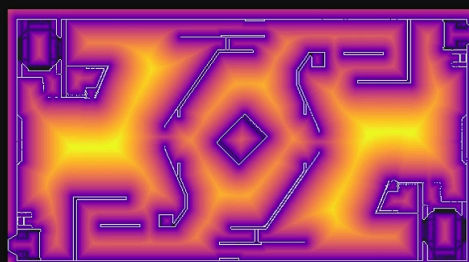
代价地图：规划器眼里的“路况”

一张栅格地图，每个格子存一个“能不能走、多危险”的数值。

到最近障碍物距离 d	格子值	含义
$d < \text{robot_radius} + \text{margin}$	100	<i>lethal</i> , 致命区域
$\text{robot_radius} + \text{margin} \leq d < d_{\text{safe}}$	$\text{round}\left(100\left(1 - e^{-w(d_{\text{safe}} - d)^2}\right)\right)$	越靠近障碍越高，把路径往中间挤
$d \geq d_{\text{safe}}$	0	自由区域，无惩罚

```

robot_radius: 0.25
margin: 0.00
d_safe: 0.60
w: 8.0
unknown_cost: 100
visualize_esdf: false
  
```



- 规划：就是在代价地图下，给出起点和终点，怎么能够去往到目标点，一般规划说的是全局，也有局部规划，

但是笔者认为局部规划更像是跟随，但是也是不分彼此了，就看每个人对此的理解

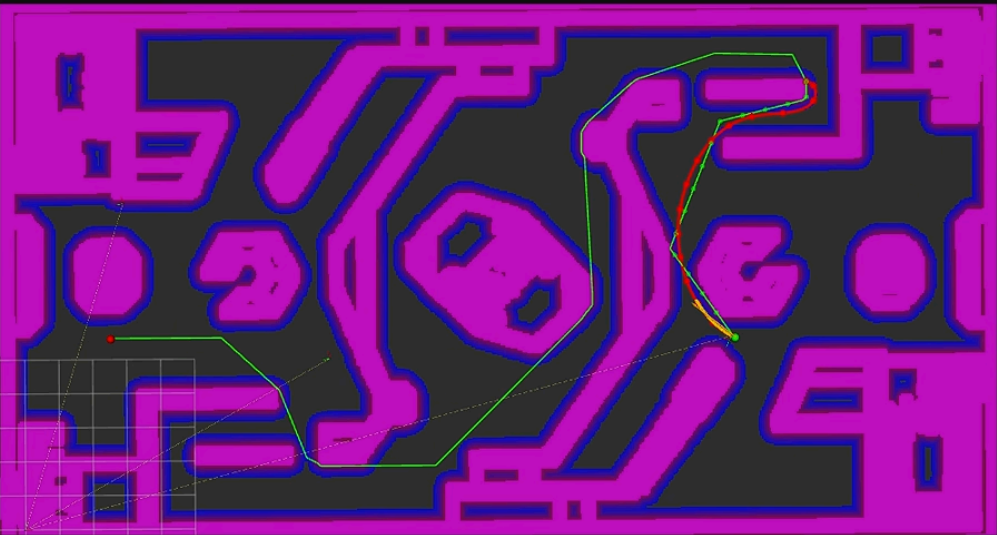
- 全局规划：一般不考虑车的相关运动学约束 (v,a 等)，只是一个搜索
- 局部规划（跟随）：更像是结合车体建模，将各种车体信息作为约束，进行规划出一条适合车体进行运动的路径——更多时候局部规划放在控制器进行优化，但是也有直接全局路径优化即直接放在全局规划器中

常见算法

算法	核心理想	特点
Dijkstra	从起点向外均匀扩散，保证最短	一定最优，但慢
A*	加启发函数 $h(n)$ ，朝目标方向优先搜	最优且快
JPS	Jump Point Search，跳过对称冗余点	A* 的加速版
Hybrid A*	把车辆运动学塞进搜索状态 (x,y,θ)	输出可行行驶曲线

全局规划与局部规划

全局规划负责找出一条大致能走的路径，局部规划让这条路径更平滑、能避障、更加符合动力学特征。



- 控制：跟上输出的路径，一般输出就是速度给下位机进行解算，目前代码是 Pure Pursuit (PID)，未来优化

方向就是 MPC，这里就是感觉到 MPC 是把局部路径优化和跟随结合到一起进行解决输出，比较优雅，但是具体是前端优化还是后段约束规划并输出，笔者能力有限尚未学习到那么多，还需后续跟进给出相关见解



控制：把期望轨迹变成速度

规划器给的是“期望速度 / 期望位姿”，控制器负责让机器人真的跟上。

算法	核心思想	特点
PID（含预瞄）	横向/航向误差做 P/I/D，常配预瞄点	实现简单、好调；高速、大曲率、有约束时容易抖或切弯
LQR	把误差线性化，二次型最优反馈 $u = -Kx$	稳、计算轻；约束（速度、加速度）只能事后裁剪
MPC	滚动优化未来一段跟踪误差，显式加 v, a, ω 约束	约束处理最好，可跟 MINCO/多项式；调参复杂
自适应 / 学习型	在线估质量、摩擦或用网络补模型误差	参数变、地面变时有用；调参和稳定性更麻烦

- 行为：上层——大脑，负责具体比赛怎么完成各种任务；下层——小脑（承接导航），告诉具体怎么到达和到达过程中出现问题如何 recovery

一般的行为决策由行为树进行整合，这里不展开赘述